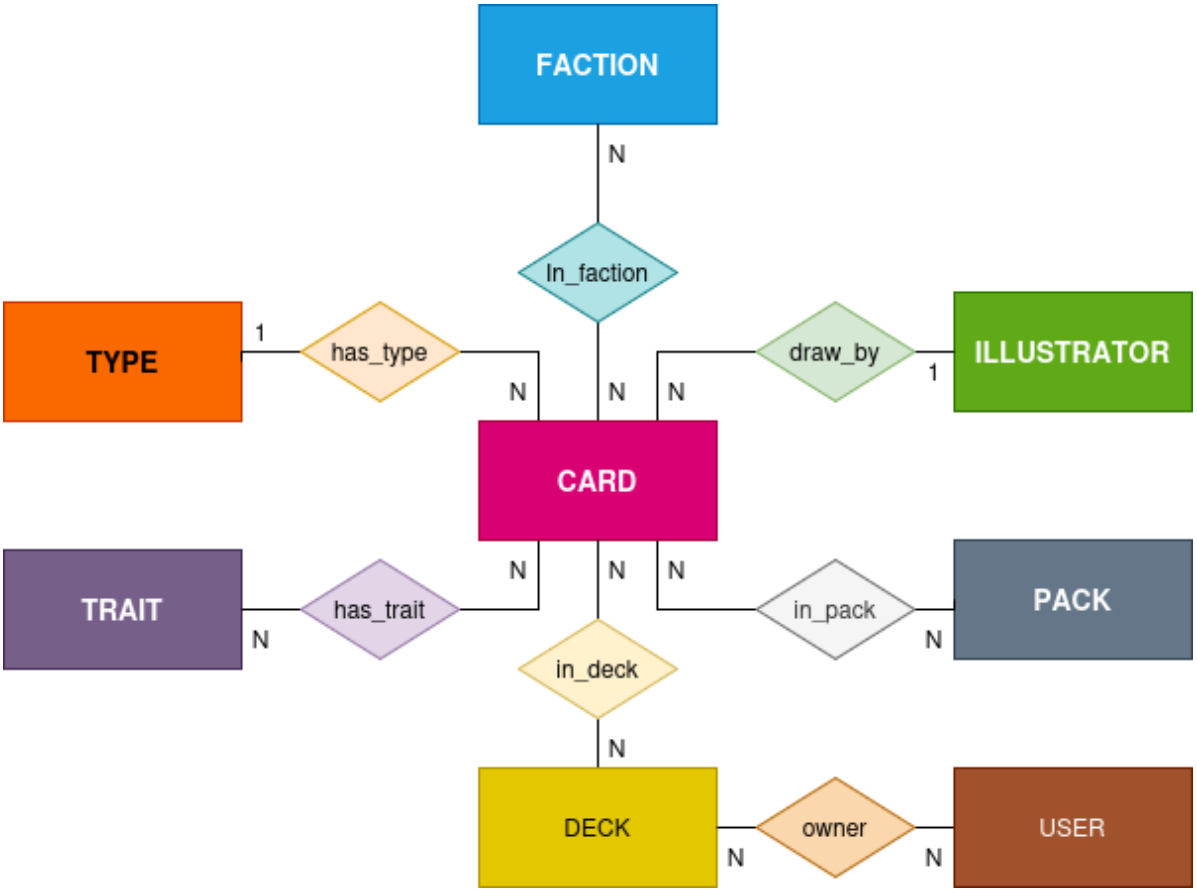
Bases de Datos II

Practica de Redis

Introducción

La empresa de Norsewind Studios tiene un juego de cartas muy popular llamado “Arkham Dread”. Con los años el número de cartas del juego comenzó a crecer y la empresa desarrolló un portal web donde los usuarios podían ver todas las cartas que hay en el juego y crear sus propios mazos. Actualmente, el portal es servido por una base de datos SQL donde se guardan todas las cartas. No obstante, desde la salida internacional del juego el portal comenzó a recibir más visitas y los tiempos de respuesta han crecido. Debajo puedes ver el esquema de la base de datos relacional.



Se pretende diseñar una cache con la información de las cartas para aliviar esa carga de la base de datos SQL y para ello se quiere hacer un pequeño prototipo usando Reddit.

Se ha decido almacenar en la cache la siguiente información de una carta:

campo	descripción
code	Código único que identifica la carta
name	Nombre de la carta
text	Texto de lo que hace la carta
type_code	Tipo de carta
traits	Keywords que describen la carta (si hay más de uno vienen serpardos por |[1])
pack_code	Expansión en la que viene la carta
faction_code	Facción de la carta, indica quien puede usar una carta (si hay más de una están separadas por |[2])
xp	Cuánto cuesta la carta
illustrator	El nombre del ilustrador de la carta
image_url	Una url con la imagen de la carta

Además, la cache debe de poder realizar las siguientes ***acciones***:

- Saber si una carta está en la cache por su campo code.
- Recuperar todos los datos de una carta a partir de su code.
- Meter una carta nueva en la cache.
- Eliminar una carta de la cache a partir de su campo code.

Objetivo I

Se pide:

1. Buscar la estructura de datos más apropiada para la cache.
2. Escribir una función Python que reciba un fichero .csv con un conjunto de ejemplo de cartas y las cargue en una base de datos Redis usando la estructura de datos seleccionada en el paso anterior.

[1] En redis cuando creamos un indice y un campo es de tipo TAG, se le puede pasar el keyword SEPARATOR para decirle que caracter se usa para separar los campos en el campo de que haya más de uno https://redis.io/docs/latest/develop/ai/search-and-query/advanced-concepts/tags/.

[2] En redis cuando creamos un indice y un campo es de tipo TAG, se le puede pasar el keyword SEPARATOR para decirle que caracter se usa para separar los campos en el campo de que haya más de uno https://redis.io/docs/latest/develop/ai/search-and-query/advanced-concepts/tags/.

3. Escribir funciones python que permita realizar cada una de las ***acciones***. Implementar una por acción.

Objetivo II

El equipo del portal ha oído hablar de las capacidades de búsqueda avanzada de Redis y quiere probar si se pueden usar para extender la funcionalidad del portal. En concreto han identificado varias búsquedas que han reclamado los usuarios a lo largo de los años:

A. En el juego hay 7 facciones, 5 que son clases que pueden usar los jugadores (mystic, survivor, guardian, seeker y rogue), la facción "neutral" que es equipo común para todas las clases y la facción "mythos" que son los enemigos. Recientemente se han añadido cartas especiales que tienen más de una facción[3]. Los jugadores están interesados en buscar todas las cartas que contengan varias facciones a la vez, así como, buscar todas las cartas que contengan al menos una de las facciones que indiquen. Por defecto devolvemos las cartas de 5 en 5.
B. Los traits son una forma muy cómoda de buscar cartas interesantes y es muy usada por los usuarios. No obstante, como hay muchos, manejarlos es algo complicado. Los usuarios quieren saber cuáles son los traits más comunes para su facción por orden de frecuencia. Por defecto mostramos páginas con 15 traits.
C. Durante el juego los jugadores ganan puntos de experiencia y pueden gastarlos en mejorar sus mazos. Por eso, es normal que los usuarios quieran buscar cartas que contengan ciertos traits y que puedan usarse para actualizar sus cartas, es decir, que tengan un coste de experiencia (xp) > 0. Normalmente están interesados en cartas que sean de su facción, así que quieren que esas aparezcan entre los primeros resultados, no obstante, también quieren ver cartas de otras facciones ya que hay mazos que mezclan varias facciones. No obstante, nunca quieren ver cartas de la facción "mythos" porque no las pueden incluir en su mazo. Los jugadores también piden filtrar las cartas por lo puntos de experiencia que les quedan, para que no les aparezcan cartas que no pueden comparar. Por defecto devolvemos las cartas de 5 en 5.

Se pide:

1. Diseña un indice que permita realizar las búsquedas anteriores.
2. Implementa una función python que realize cada una de las búsquedas anteriores.
3. ¿Cómo comprobarias que has implementado correctamente las búsquedas anteriores?

[3] Las cratas multifacción nunca pueden contener la facción Neutral ni Mythos.

Objetivo III

El equipo del portal quiere implementar un recomendador automático de cartas. La idea es que, dada una carta, el sistema muestre otras 5 cartas parecidas a dicha carta que a los usuarios les pudieran interesar. Para ello, se ha consultado con el equipo de diseño de "Arkham Dread" y han dicho que lo mejor sería utilizar el nombre y el texto de la carta, si no fuera posible también indican que los traits son una alternativa, aunque son menos específicos generales. Al preguntarles por las ilustraciones, el equipo de diseño ha respondido que están más pensadas como decoración y no dan información sobre lo que hace la carta.

Con esta información el equipo de desarrollo ha planteado 4 alternativas de más sencilla a más compleja:

A. Usar solo metadatos de la carta y obviar el nombre y el texto: Proponen buscar cartas de la misma facción y que tengan alguno de los traits de la carta sobre la que se realiza la búsqueda.
B. Capacidades full-text de redis: Aprovechar las capacidades de redis y hace una busqueda full-text con el texto de la carta.
C. Búsqueda semántica: Hacer embeddings con el nombre y el texto de la carta para luego hacer búsqueda semántica.

Se pide:

1. Diseña un índice que se pueda usar en cualquiera de las alternativas anteriores.
2. Haz una función python que implemente cada una de las alternativas propuestas. La función debe recibir el código de una carta e imprimir por pantalla 5 otras cartas parecidas[4].
3. Comprueba los resultados obtenidos ¿Qué diferencias hay entre las distintas alternativas? ¿Cuántas cartas es capaz de devolver cada aproximación? ¿Qué tal es el orden que devuelve? Da tu opinión justificada en los resultados obtenidos. Haz las pruebas usando la carta con código 1029 (shotgun).

[4] Puedes hacer más de una consulta a la base de datos si lo consideras necesario.